ResearchOps AI — DevOps FA2 Engineering Report

Course: PEC 4 - DevOps (Units 3 & 4) | **Evaluation:** FA2 Activity (10 Marks) | **Target:** October 2026

1. Project Overview & Working

ResearchOps AI is an asynchronous literature intelligence microservice and DevOps platform. It solves the real-world problem of scattered academic research by concurrently querying three open-access registries (**Wikipedia**, **OpenAlex**, and **Crossref**), automatically deduplicating DOIs, and providing grounded RAG literature review generation, while running on an observable, resilient microservice architecture.

End-to-End Workflow:

- **1. User Query:** User submits a topic (e.g. *'Quantum Computing'*) via the Single-Page Web UI.
  - **2. Async Aggregation:** FastAPI initiates concurrent httpx queries via asyncio.gather to Wikipedia, OpenAlex, and Crossref.
  - **3. Deduplication & Synthesis:** Normalizes DOIs and publication titles across registries, eliminating duplicate citations.
  - **4. State & Background Jobs:** Results persist in **PostgreSQL 16**, are cached in **Redis 7**, and heavy PDF chunking runs via **Celery**.
  - **5. SRE Telemetry:** Every request live-updates custom **Prometheus** counters and gauges (latency, traffic, errors, saturation).

Microservice Architecture Topology:

Service	Technology	Role & Responsibilities	Port
API	FastAPI (Python 3.12)	Core REST API, async provider orchestrator, endpoints (/research, /health, /metrics)	8000
Worker	Celery + Redis	Asynchronous document parsing, PDF chunking, 384-dim dense embeddings	Internal
Database	PostgreSQL 16 (pgvector)	Relational store for saved research, projects, chats, and audit logs	5432
Broker	Redis 7 Alpine	In-memory task broker and caching layer coordinating async jobs	6379
Prometheus	Prometheus v2.51	Scrapes /metrics every 15s; evaluates provider latency and saturation	9090
Grafana	Grafana v10.4	Visual telemetry dashboard tracking the 4 Golden Signals and SLO dials	3001
Gateway	Nginx Alpine	Reverse proxy gateway routing traffic to API and frontend assets	8080

2. DevOps Implementation Breakdown (Units 3 & 4)

Below is the exact engineering breakdown of how and why each DevOps stage is implemented in this codebase:

1. Source Code & Quality Gates (Git, Ruff, Pytest)

Why: Guarantees zero code regressions, enforces clean code hygiene, and validates endpoint contracts.
How: Configured ruff linter for instant linting; authored **35 automated tests** in tests/ covering health probes, provider fallback mocking, deduplication logic, and Prometheus metrics formatting.
Command: ruff check app tests | pytest -v --cov=app

2. Continuous Integration (GitHub Actions)

Why: Eliminates manual testing. Every commit is tested and verified on a clean Ubuntu runner in the cloud.
How: Workflow in .github/workflows/ci.yml runs on push/PR. Stages: (1) Setup Python 3.12 with pip cache; (2) Ruff static analysis; (3) Pytest suite (all 35 tests must pass); (4) Docker Buildx building both API and Worker images.
Status: Automated green checkmark on GitHub on every commit.

3. Containerization (Docker & Multi-Stage Builds)

Why: Solves environment drift and guarantees identical behavior across developer laptops, cloud, and Kubernetes.
How: Dockerfile uses python:3.12-slim base (~200MB), runs as non-root user appuser for security hardening, and embeds an internal HEALTHCHECK probing /health.
Command: docker build -t researchops-api:latest .

4. Multi-Container Orchestration (Docker Compose)

Why: Spins up the full 6-service microservice stack locally with a single command without port clashes.
How: docker-compose.yml connects API, Worker, PostgreSQL, Redis, Prometheus, Grafana, and Nginx on an internal bridge network with named volumes (pg_data, redis_data, prom_data).
Command: docker compose up --build -d | docker compose ps

5. Infrastructure as Code (Terraform on AWS)

Why: Eliminates error-prone manual cloud console clicking; infrastructure is version-controlled and reproducible.
How: terraform/main.tf provisions an AWS EC2 instance (Ubuntu 22.04 LTS) and Security Group opening ports 22, 80, 8000, 9090, and 3001, outputting the public IP automatically.
Command: cd terraform && terraform init && terraform plan && terraform apply

6. Configuration Management (Ansible)

Why: Connects to the raw EC2 instance over SSH and turns it into a running platform with zero manual SSH commands.
How: ansible/deploy.yml updates OS packages, installs Docker Engine & Compose, clones repository to /opt/researchhub, templates .env, and runs docker compose up -d.
Command: ansible-playbook -i ansible/inventory.ini ansible/deploy.yml

7. Production Container Orchestration (Kubernetes & Kustomize)

Why: Enterprise high availability, zero-downtime rolling updates, self-healing pods, and autoscaling.
How: Suite of 11 manifests in k8s/: Namespace, ConfigMap, Secret, StatefulSets for Postgres & Redis with PVCs, Deployment with 2 replicas and RollingUpdate (maxSurge: 1, maxUnavailable: 0), Ingress routing, and **HPA** scaling pods up to 8 replicas when CPU exceeds 70%. Bundled with kustomization.yaml.
Command: kubectl apply --dry-run=client -k k8s/ | kubectl apply -k k8s/

8. Monitoring, Observability & SRE (Prometheus & Grafana - Unit 4)

Why: Adheres to Google SRE principles: monitoring health before users complain, measuring Golden Signals and SLOs.
How: Implemented distinct /health (Liveness) and /ready (Readiness) probes. Exposes custom /metrics in Prometheus exposition format. Tracks probe latencies for OpenAlex, Wikipedia, and Crossref. Grafana dashboard auto-provisioned at port 3001. Enforces **99.5% availability SLO**.
Command: curl http://localhost:8000/metrics | http://localhost:3001

3. SRE Framework: 4 Golden Signals & SLO Alignment

Golden Signal	Implemented Metric / Probe	Engineering Purpose	Target SLO / Threshold
1. Latency	researchops_provider_latency_ms	Measures real-time response times of OpenAlex, Wikipedia, and Crossref probes.	p95 < 1500 ms
2. Traffic	researchops_total_papers researchops_total_projects	Measures active system usage demand, database throughput, and saved literature items.	> 100 concurrent reqs
3. Errors	Probe degradation status & HTTP 5xx tracking	Tracks rate of requests that fail or encounter third-party upstream API timeouts.	< 0.5% failure rate (99.5% uptime SLO)
4. Saturation	researchops_active_background_jobs	Monitors Celery worker queue depth, CPU limits, and DB connection pool saturation.	Queue depth < 20 CPU < 70%

4. Faculty Viva Defense — Top 5 Questions & Crisp Answers

Q1: Why did you separate /health (Liveness) and /ready (Readiness) probes?

Answer: A Liveness probe checks if the Python/FastAPI process is executing; if it fails, the orchestrator (Kubernetes/Docker) restarts the pod. A Readiness probe verifies that downstream dependencies (PostgreSQL and Redis) are connected. If the DB is saturated, the readiness probe fails, instructing Kubernetes to temporarily stop routing incoming traffic to that pod without needlessly killing and restarting the process.

Q2: How does your project implement Unit 4 SRE Golden Signals?

Answer: We expose custom metrics in /metrics: (1) **Latency** via provider probe latencies; (2) **Traffic** via total projects/papers; (3) **Errors** via probe degradation status; and (4) **Saturation** via Celery background job queue depth and database connection counts.

Q3: What is the architectural distinction between Terraform and Ansible?

Answer: Terraform is our *Infrastructure as Code (IaC)* tool that provisions the AWS virtual machine and security firewall rules. Ansible is our *Configuration Management* tool that SSH-connects to that VM to install Docker, configure environment secrets, and launch the multi-service container cluster.

Q4: How does Kubernetes guarantee zero-downtime deployments in your manifests?

Answer: In k8s/05-api-deployment.yaml, we enforce a RollingUpdate strategy with maxSurge: 1 and maxUnavailable: 0. Kubernetes spins up a new pod, waits for its readiness probe to return healthy, and only then tears down the old replica.

Q5: How does the system handle an external provider outage (e.g. Crossref or Wikipedia down)?

Answer: The orchestrator executes parallel async calls with asyncio.gather(..., return_exceptions=True). If an external API times out, it is caught gracefully: the user receives partial synthesized results with a warning badge, Prometheus records degraded status, and the application never crashes.

5. Quick Verification Commands

Verification Step	Command to Execute	Expected Result
1. Code Linter	`ruff check app tests`	0 errors / clean code hygiene
2. Test Suite	`pytest -v --cov=app`	35 / 35 tests passed in ~8 seconds
3. Full Local Stack	`docker compose up --build -d`	All 6 microservices running and healthy
4. Automated DoD Check	`python scripts/verify_fa2.py`	Full green readiness scorecard
5. K8s Manifest Dry-Run	`kubectl apply --dry-run=client -k k8s/`	All 11 manifests pass validation